

Transferring Secrets Between CircleCI Organizations

A guide to moving context and project environment variables org-to-org — without writing secrets to disk

CircleCI
2026

Transferring Secrets Between CircleCI Organizations

Audience: CircleCI organization administrators moving environment variables from one CircleCI organization to another (for example, during a GitHub organization rename or consolidation).

Scope: This guide covers **secrets movement only** — context environment variables and project-level environment variables. It does not cover migrating projects, pipelines, or org settings. (For a full migration, see the companion *GitHub OAuth → OAuth Migration Guide*.)

Tooling: the `circleci-migrate` CLI, `secrets transfer` command.

Why a CLI and not copy/paste? CircleCI never returns a secret **value** through its API — values are write-only by design. The only place a value can be read is *inside a running pipeline* of the project or organization that owns it. `secrets transfer` uses this fact to move values securely, in-flight, without ever writing them to a file, an artifact, or external storage.

Prerequisite — destination projects must already exist. To transfer **project** environment variables, the matching destination projects must already be **onboarded/created** in the destination org; the transfer skips any project it can't map to an existing destination project. (Destination **contexts** are created automatically; **projects** are not.) Run your `org / project / context sync` first — or follow the *GitHub OAuth → OAuth Migration Guide* — before transferring project env vars. Context-only transfers have no such requirement.

1. How it works (the security model)

`secrets transfer` runs entirely **inside your source organization's own CI**:

1. You run the CLI from your laptop. It does **not** read any secret values locally.
2. The CLI submits a short-lived, inline pipeline to your source org.
3. Inside that pipeline, CircleCI injects the real values (contexts are attached to the job; project variables are injected into that project's own pipeline).
4. The job sends each value **directly** to the destination organization's API over TLS, then exits.

No secret value ever touches disk, a build artifact, or external storage.

The destination token is never embedded

The in-pipeline job needs an API token for the **destination** org. You never put that token in any file or config. Instead:

1. You store the destination token in a **context in your source org** (for example, a context named `migration-secrets` containing a variable `CIRCLECI_DEST_TOKEN`).
2. You tell the CLI the **name** of that context.
3. CircleCI injects the token into the job at runtime; the generated pipeline config references it only as `${CIRCLECI_DEST_TOKEN}` — the literal value never appears anywhere.

Trust note: anyone who can create pipelines in the source org can already read its contexts. Use a destination token scoped to the minimum needed (context/project write), and **rotate it after the transfer completes**.

2. What transfers, and what doesn't

Item	Transfers with <code>secret transfer</code> ?	Notes
Context environment variables	 Yes	One pipeline carries all contexts. Missing destination contexts are created automatically .
Project environment variables	 Yes (opt-in: <code>--include-project-vars</code>)	One pipeline per project (see §5). Destination project must already exist.
Additional project SSH keys		

Item	Transfers with <code>secret</code> transfer?	Notes
	 Yes (opt-in: <code>--include-ssh-keys</code>)	Same in-pipeline zero-disk path as project env vars. Requires <code>--mapping</code> . Checkout / deploy keys are re-created automatically when you follow the project.
Restricted contexts	 Skipped by default	See §6.

3. Prerequisites

Before you begin, confirm each of the following.

- ☐ **Both organizations exist** in CircleCI and use the GitHub OAuth integration (slugs look like `gh/your-org`).
- ☐ You are an **organization admin** of *both* the source and destination orgs.
- ☐ **Two personal API tokens** (User Settings → Personal API Tokens), one whose user admins the source org and one for the destination org.
- ☐ The source org has **"unversioned config" enabled** (Organization Settings → Advanced → *Allow API-triggered pipelines to use unversioned config*). The CLI will detect this and offer to enable it if it is off.
- ☐ The destination organization's **Organization ID** (a UUID). Find it in the destination org's **Organization Settings → Overview**.
- ☐ **(Project env vars only)** The destination **projects are already onboarded/created** in the destination org. The transfer auto-creates destination *contexts* but **not** projects, and skips any project with no existing destination counterpart. Run the org/project/context sync (or the *GitHub OAuth → OAuth Migration Guide*) first. Context-only transfers do not need this.
- ☐ The `circleci-migrate` CLI installed:

```
brew install AwesomeCICD/tap/circleci-migrate
circleci-migrate version    # confirm v0.17.1 or later
```

Set your tokens once for the session:

```
export CIRCLECI_SOURCE_TOKEN="<source-org-admin-token>"
export CIRCLECI_DEST_TOKEN="<destination-org-admin-token>"
```

4. Transferring context environment variables

This is the simplest path and the most common need.

Step 4.1 — Export the source org

`export` is **read-only**. It records the *names* of your contexts, projects, and variables (never the values) into a manifest the transfer command reads.

```
circleci-migrate export \
  --source-org gh/acme \
  --output manifest.json
```

Step 4.2 — Store the destination token in a source-org context

In the **source** org's UI (Organization Settings → Contexts), create a context named `migration-secrets` and add one variable:

Variable	Value
<code>CIRCLECI_DEST_TOKEN</code>	<i>the destination org's API token</i>

Step 4.3 — Dry run (no changes)

Always preview first. Without `--apply`, nothing is triggered or written — the command prints exactly what it *would* do.

```
circleci-migrate secrets transfer \
  --manifest manifest.json \
  --dest-org-id <destination-org-uuid> \
  --dest-token-context migration-secrets
```

Review the plan: each context, how many variables, and whether the destination context would be **created** or **updated**.

Step 4.4 — Apply

```
circleci-migrate secrets transfer \
  --manifest manifest.json \
  --dest-org-id <destination-org-uuid> \
```

```
--dest-token-context migration-secrets \  
--enable-trigger \  
--apply
```

This triggers one pipeline in the source org that reads each context's values and writes them to the matching destination context (creating it if needed).

`--enable-trigger` lets the CLI enable the "unversioned config" org setting if it is currently off. Omit it if you have already enabled it manually.

To transfer **only specific contexts**, repeat `--context` :

```
circleci-migrate secrets transfer ... --context deploy-prod --context shared --  
apply
```

5. Transferring project environment variables

Project variables need one extra concept and one extra flag.

Why it's different

CircleCI **project** environment variables are *project-scoped*: their values are only readable inside **that project's own** pipeline. So the CLI runs **one pipeline per source project**, each under its own project, so every project's values are injected correctly. These run in parallel.

The destination **project must already exist** (be onboarded) in the destination org. And the CLI needs to know which source project maps to which destination project.

Step 5.1 — Generate the project mapping

The `mapping generate` command matches your source projects to the destination org's projects **by repository name** and writes a `mapping.json`, plus a report of anything that didn't match.

```
circleci-migrate mapping generate \  
--manifest manifest.json \  
--dest-org gh/acme-new \  
-o mapping.json
```

The report has three sections:

- **Matched** — written to `mapping.json` (e.g. `gh/acme/web` → `gh/acme-new/web`).

- **Unmatched source** — a source project with no destination counterpart. Onboard that repo in the destination org first, or add a manual entry to `mapping.json`.
- **Destination-only** — destination projects with no source match (just informational).

Review `mapping.json` and the report before continuing.

Step 5.2 — Dry run with project vars

```
circleci-migrate secrets transfer \
  --manifest manifest.json \
  --dest-org-id <destination-org-uuid> \
  --dest-token-context migration-secrets \
  --mapping mapping.json \
  --include-project-vars
```

The plan now also lists, per project, which variables would transfer and which projects are **skipped** (no mapping entry → onboard them first).

Step 5.3 — Apply

```
circleci-migrate secrets transfer \
  --manifest manifest.json \
  --dest-org-id <destination-org-uuid> \
  --dest-token-context migration-secrets \
  --mapping mapping.json \
  --include-project-vars \
  --enable-trigger \
  --apply
```

The summary reports how many context and project pipelines succeeded.

6. Restricted contexts and SSH keys

Restricted contexts

Project-type and expression restrictions block the transfer pipeline by default when the host project is not in the allowed set — the workflow returns “unauthorized.”

Context restrictions and sync : when you run the full migration (`sync --apply`), **project-type restrictions are remapped and recreated automatically** on the destination via the project mapping. **Group** restrictions require manual recreation in the destination UI — group IDs differ between orgs and cannot be remapped automatically.

Default behavior (no flag): the CLI detects blocking restrictions from the manifest at plan time and fails fast with an actionable error listing the affected contexts and instructions to re-run with `--remove-restrictions`.

With `--remove-restrictions`: the CLI temporarily removes project/expression restrictions from the source context, triggers the transfer pipeline, then restores them afterwards (best-effort). Group restrictions (including the default “All members”) are **never** touched.

```
circleci-migrate secrets transfer \  
  --manifest manifest.json \  
  --dest-org-id <destination-org-uuid> \  
  --dest-token-context migration-secrets \  
  --remove-restrictions \  
  --enable-trigger \  
  --apply
```

If restore fails (e.g. due to a transient error), the CLI prints a `WARNING` with the exact restriction to re-add manually — no restriction is silently lost.

SSH keys

`--include-ssh-keys` transfers **additional project SSH keys** via the same in-pipeline zero-disk path: the job uses `add_ssh_keys` to materialize each key, reads the private key with `jq --rawfile` (never echoed to logs), and POSTs it to the destination project. Requires a `--mapping` entry for each project.

Checkout / deploy keys are *not* moved by `secrets transfer`; these are re-created automatically when you follow the project in the destination org. Re-add any manually-uploaded deploy keys in the destination project settings.

7. Verify

In the **destination** org UI:

- **Organization Settings → Contexts** — every expected context is present with the expected number of variables.
- **Project Settings → Environment Variables** — for a sample of projects, variable **names** are present (values are masked, as always).

For a structured parity check, use the `validate` command — it exports both orgs read-only and prints a per-section report (Contexts, Projects, Org Settings, Runners, Orbs, CIAM) with ✓ matched / × missing / ▲ manual items, a NEEDS ATTENTION summary, and an exit code of 1 if anything is missing:

```
circleci-migrate validate \  
  --source-org gh/acme \  
  --dest-org gh/acme-new \  
  --mapping mapping.json
```

Note: secret *values* are never compared — the API masks them. `validate` checks presence and structure (names, counts, settings, restrictions).

Alternatively, a quick manual cross-check:

```
circleci-migrate export \  
  --source-org gh/acme-new \  
  --source-token "$CIRCLECI_DEST_TOKEN" \  
  --output manifest-dest.json
```

Compare context and project variable counts between `manifest.json` and `manifest-dest.json`.

8. Security checklist (do this after transfer)

☐

Confirmed destination contexts/projects have the expected variables.

☐

Rotated the destination API token stored in `migration-secrets`.

☐

Deleted the `migration-secrets` context from the source org (or rotated its token) once the transfer is complete.

☐

Considered rotating any high-sensitivity values as a hygiene step.

☐

Used a **private** project as the host for the transfer pipeline.

`secrets transfer` never writes values to disk or artifacts, so there is no local file to delete — one of its key advantages.

9. Troubleshooting

Symptom	Cause / fix
Permission denied mid-run / pipeline skipped	"Unversioned config" is off in the source org. Re-run with <code>--enable-trigger</code> , or enable it in Org Settings → Advanced.
A project shows SKIP ... dest project unknown	No mapping entry. Run <code>mapping generate</code> , or add the entry to <code>mapping.json</code> ; onboard the repo in the destination first if needed.
Project variables transferred as empty	You are on an older CLI. Upgrade to v0.17.1+ , which runs one pipeline per project so each project's values inject correctly.
<code>--dest-org-id</code> rejected	You passed a slug. This flag needs the destination Organization ID (UUID) from Org Settings → Overview.
Destination context not created	The destination org must exist and the token must be an org admin token.

Command quick reference

```
# 1. Export source (read-only)
circleci-migrate export --source-org gh/acme --output manifest.json

# 2. (project vars / SSH keys only) Generate the mapping
circleci-migrate mapping generate --manifest manifest.json --dest-org gh/acme-new
-o mapping.json

# 3. Dry run
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <uuid> --dest-token-context migration-secrets \
  [--mapping mapping.json] [--include-project-vars] [--include-ssh-keys]

# 4. Apply
circleci-migrate secrets transfer --manifest manifest.json \
  --dest-org-id <uuid> --dest-token-context migration-secrets \
  [--mapping mapping.json] [--include-project-vars] [--include-ssh-keys] \
  [--remove-restrictions] [--host-project gh/acme/<repo>] \
  --enable-trigger --apply

# 5. Verify
circleci-migrate validate --source-org gh/acme --dest-org gh/acme-new --mapping
mapping.json
```

© CircleCI. Generated for customer use. Commands assume `circleci-migrate` v0.17.1 or later.